Contents

Step 2.0 – Creating a New Spring Boot Maven Project 1

Step 2.1 – Creating a Spring MVC web controller 3

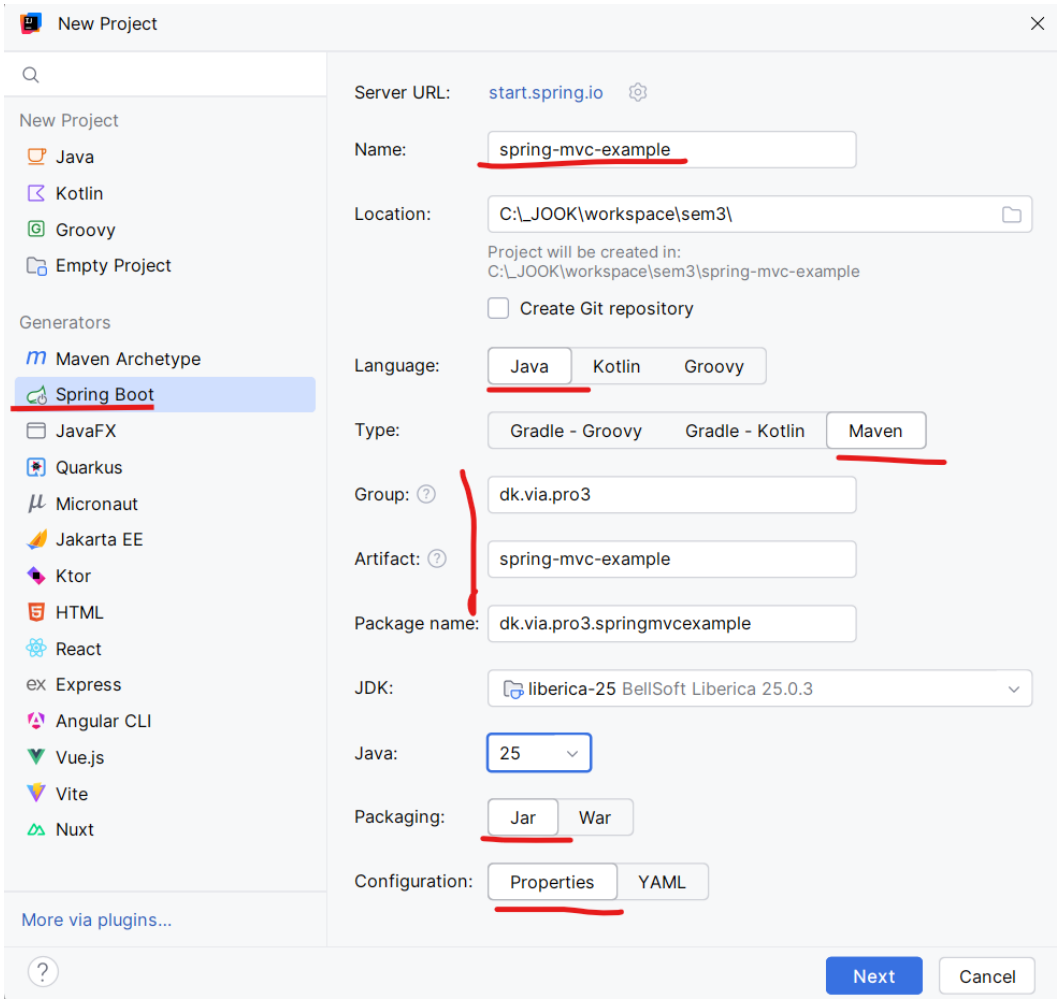
Step 2.2 – Creating the content using templates 3

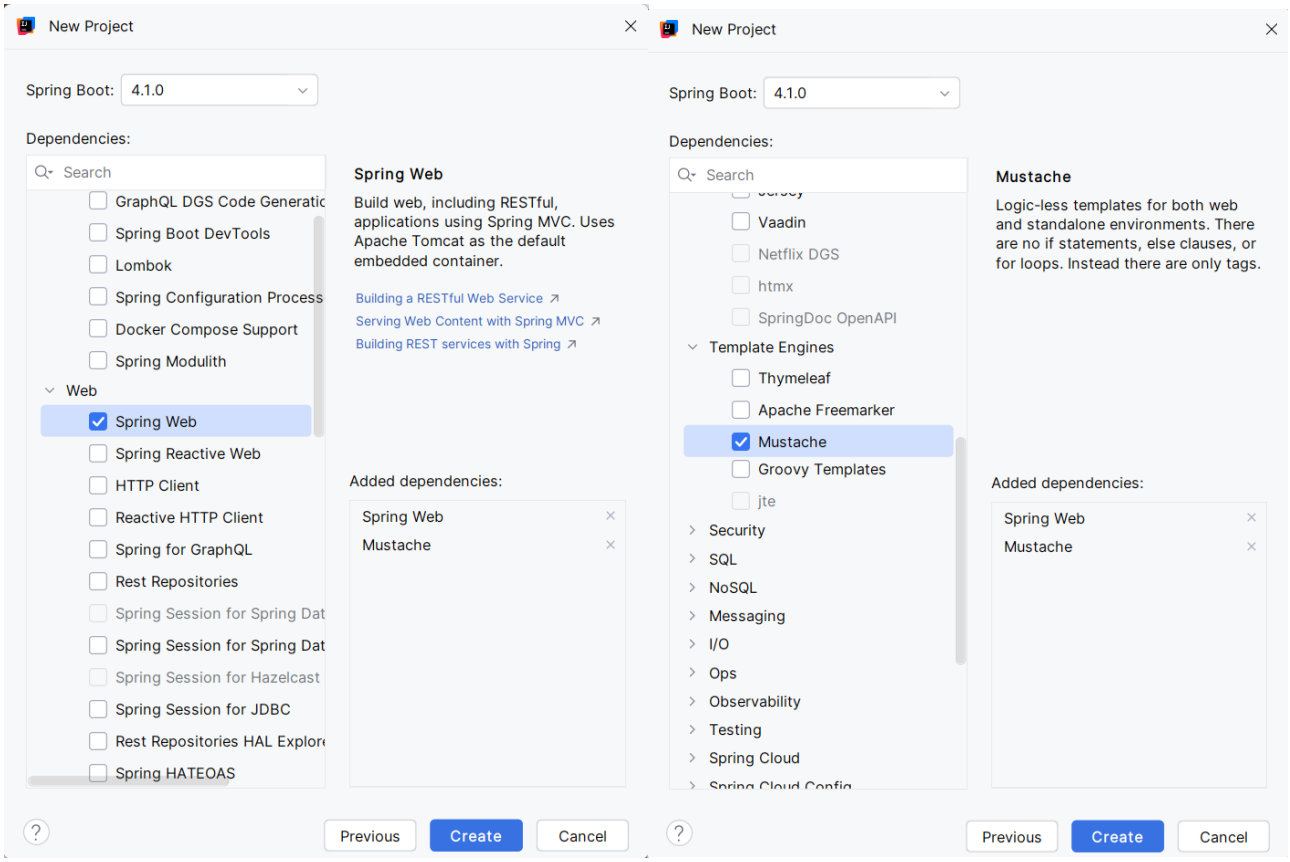
Step 2.3 – Run the application 3

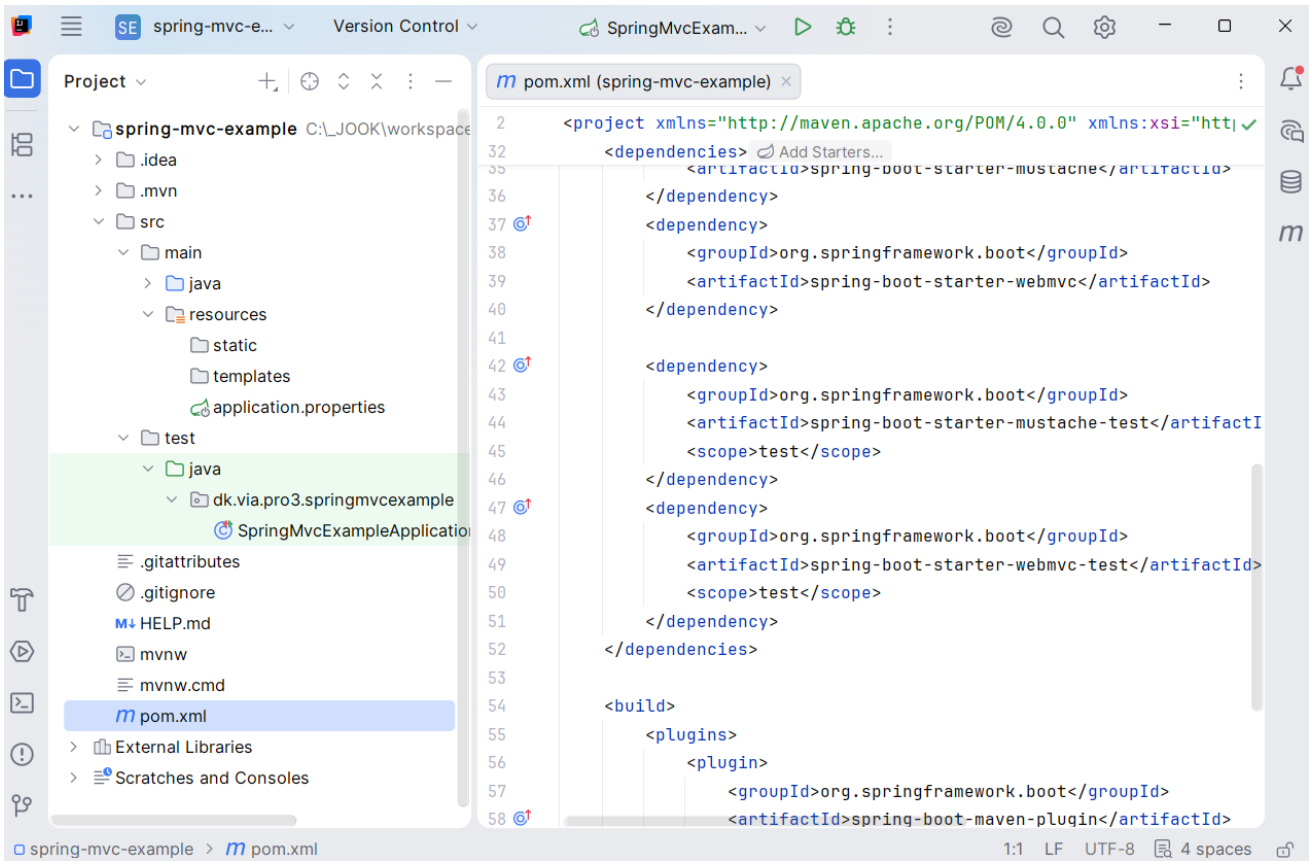
In this set of exercises, we'll practice the basics of creating a web application using Spring Boot. We will explore the use of Maven and IntelliJ when working with Spring Boot. We'll also look briefly at Mustache templating engine.

Step 2.0 – Creating a New Spring Boot Maven Project

Start IntelliJ and create a New Spring Boot project. (Maven. Java, Jar, properties)




New Project
Spring Boot: 4.1.0
Dependencies:
Search
GraphQL DGS Code Generatic
Spring Boot DevTools
Lombok
Spring Configuration Process
Docker Compose Support
Spring Modulith
Web
Spring Web
Spring Reactive Web
HTTP Client
Reactive HTTP Client
Spring for GraphQL
Rest Repositories
Spring Session for Spring Dat
Spring Session for Spring Dat
Spring Session for Hazelcast
Spring Session for JDBC
Rest Repositories HAL Explor
Spring HATEOAS
Spring Web
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
Building a RESTful Web Service
Serving Web Content with Spring MVC
Building REST services with Spring
Added dependencies:
Spring Web
Mustache
Previous
Create
Cancel
New Project
Spring Boot: 4.1.0
Dependencies:
Search
Vaadin
Netflix DGS
htmx
SpringDoc OpenAPI
Template Engines
Thymeleaf
Apache Freemarker
Mustache
Groovy Templates
jte
Security
SQL
NoSQL
Messaging
I/O
Ops
Observability
Testing
Spring Cloud
Mustache
Logic-less templates for both web and standalone environments. There are no if statements, else clauses, or for loops. Instead there are only tags.
Added dependencies:
Spring Web
Mustache
Previous
Create
Cancel


spring-mvc-e...
Version Control
SpringMvcExam...
Project
spring-mvc-example C:_JOOK\workspace
.idea
.mvn
src
main
java
resources
static
templates
application.properties
test
java
dk.via.pro3.springmvcexample
SpringMvcExampleApplicatio
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
pom.xml (spring-mvc-example)
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="htt
<dependencies>
Add Starters...
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-mustache-test</artifactI
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc-test</artifactId>
<scope>test</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
spring-mvc-example > pom.xml
1:1 LF UTF-8 4 spaces

Step 2.1 – Creating a Spring MVC web controller

Spring MVC is Spring framework's module that allows us to build web apps on top of servlet-based containers using the MVC pattern, which typically returns HTML content.

For instance:

```
package dk.via.pro3.springmvcexample.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MainController {
   @GetMapping("/")
   public String index() {
      return "index";
   }
}
```

@Controller: Spring MVC's annotation shows that this class is a web controller.

@GetMapping: Spring MVC's annotation to map HTTP GET/calls to methods (view) in this example – index (the name of template to render). Recall the mustache template engine we added to the dependencies.

Step 2.2 – Creating the content using templates

By default, Spring Boot expects all templates to be put in src/main/resources/templates folder.

Create index.mustache file in the templates folder and add the following code:

```
<h1>Programming 3 - Distributed Systems</h1>
<p>
   In this course, we'll work with distributed systems and
    explore the various middlewares such as <b>gRPC, Spring and RabbitMQ</b>.
</p>
```

Step 2.3 – Run the application

Run the application and navigate to localhost:8080 in your browser to see the result.

